# Telematics Platform User Guide

Modem Configuration Manager for ConMod

| Document Information | |
|---|---|
| Filename | TP_F100U-00044_User_Guide_Modem_Configuration_Manager_ConMod |
| Status | Release |
| Revision Date | Feb 16, 2023 |
| Version | 03.00 |
| Document ID | TP_F100U-00044 |


**Standard Proprietary Terms**

THE REPRODUCTION, DISTRIBUTION AND UTILIZATION OF THIS DOCUMENT AS WELL AS THE COMMUNICATION OF ITS CONTENTS TO OTHERS WITHOUT EXPRESS AUTHORIZATION IS PROHIBITED. OFFENDERS WILL BE HELD LIABLE FOR THE PAYMENT OF DAMAGES. ALL RIGHTS RESERVED IN THE EVENT OF THE GRANT OF A PATENT, UTILITY MODEL OR DESIGN.





© Continental Automotive GmbH 2019
Confidential and Proprietary

# Table of contents

**1 Introduction ..... 6**
**1.1 Notational Conventions ..... 7**
**2 Setup ..... 7**
**2.1 Tools/Equipment Needed ..... 7**
**2.2 Setup Instructions ..... 7**
2.2.1 Hardware Setup Instructions ..... 8
**2.3 Setup Troubleshooting ..... 8**
**3 Features/Components ..... 8**
**3.1 Client registration ..... 8**
3.1.1 Overview ..... 8
3.1.1.1 Platform Content Overview ..... 8
3.1.1.2 Product Expectations ..... 8
3.1.1.3 Extending Platform Functionality ..... 8
3.1.2 Setup ..... 8
3.1.2.1 Tools/Equipment Needed ..... 8
3.1.2.2 Setup Instructions ..... 8
3.1.2.3 Setup Troubleshooting ..... 8
3.1.3 Usage ..... 9
3.1.3.1 Basic Instructions ..... 9
3.1.3.2 Detailed Instructions ..... 9
3.1.3.3 Code Example ..... 9
3.1.3.4 Testing/Error Reporting ..... 10
**3.2 Auto-selection of Modem Configurations ..... 10**
3.2.1 Overview ..... 10
3.2.1.1 Platform Content Overview ..... 10
3.2.1.2 Product Expectations ..... 11
3.2.1.3 Extending Platform Functionality ..... 11
3.2.2 Setup ..... 11
3.2.2.1 Tools/Equipment Needed ..... 11
3.2.2.2 Setup Instructions ..... 11
3.2.2.3 Setup Troubleshooting ..... 11
3.2.3 Usage ..... 11
3.2.3.1 Basic Instructions ..... 11
3.2.3.2 Detailed Instructions ..... 11
3.2.3.3 Testing/Error Reporting ..... 11
**3.3 Get Modem configuration List ..... 11**
3.3.1 Overview ..... 11
3.3.1.1 Platform Content Overview ..... 11
3.3.1.2 Product Expectations ..... 11


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.3.1.3 Extending Platform Functionality ..... 12
3.3.2 Setup ..... 12
3.3.2.1 Tools/Equipment Needed ..... 12
3.3.2.2 Setup Instructions ..... 12
3.3.2.3 Setup Troubleshooting ..... 12
3.3.3 Usage ..... 12
3.3.3.1 Basic Instructions ..... 12
3.3.3.2 Detailed Instructions ..... 12
3.3.3.3 Code Example ..... 12
3.3.3.4 Testing/Error Reporting ..... 13
**3.4 Delete Modem Configuration ..... 14**
3.4.1 Overview ..... 14
3.4.1.1 Platform Content Overview ..... 14
3.4.1.2 Product Expectations ..... 14
3.4.1.3 Extending Platform Functionality ..... 14
3.4.2 Setup ..... 14
3.4.2.1 Tools/Equipment Needed ..... 14
3.4.2.2 Setup Instructions ..... 14
3.4.2.3 Setup Troubleshooting ..... 14
3.4.3 Usage ..... 14
3.4.3.1 Basic Instructions ..... 14
3.4.3.2 Detailed Instructions ..... 14
3.4.3.3 Code Example ..... 14
3.4.3.4 Testing/Error Reporting ..... 15
**3.5 Manual Modem Configuration selection ..... 15**
3.5.1 Overview ..... 15
3.5.1.1 Platform Content Overview ..... 15
3.5.1.2 Product Expectations ..... 16
3.5.1.3 Extending Platform Functionality ..... 16
3.5.2 Setup ..... 16
3.5.2.1 Tools/Equipment Needed ..... 16
3.5.2.2 Setup Instructions ..... 16
3.5.2.3 Setup Troubleshooting ..... 16
3.5.3 Usage ..... 16
3.5.3.1 Basic Instructions ..... 16
3.5.3.2 Detailed Instructions ..... 16
3.5.3.3 Testing/Error Reporting ..... 19
**3.6 Software Loading Support ..... 19**
3.6.1 Overview ..... 19
3.6.1.1 Platform Content Overview ..... 19
3.6.1.2 Product Expectations ..... 19
3.6.1.3 Extending Platform Functionality ..... 19
3.6.2 Setup ..... 19


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.6.2.1 Tools/Equipment Needed ..... 19
3.6.2.2 Setup Instructions ..... 19
3.6.2.3 Setup Troubleshooting ..... 19
3.6.3 Usage ..... 20
3.6.3.1 Basic Instructions ..... 20
3.6.3.2 Detailed Instructions ..... 20
3.6.3.3 Testing/Error Reporting ..... 20
**3.7 Get MCFG state change information ..... 20**
3.7.1 Overview ..... 20
3.7.1.1 Platform Content Overview ..... 20
3.7.1.2 Product Expectations ..... 20
3.7.1.3 Extending Platform Functionality ..... 20
3.7.2 Setup ..... 20
3.7.2.1 Tools/Equipment Needed ..... 20
3.7.2.2 Setup Instructions ..... 20
3.7.2.3 Setup Troubleshooting ..... 20
3.7.3 Usage ..... 21
3.7.3.1 Basic Instructions ..... 21
3.7.3.2 Detailed Instructions ..... 21
3.7.3.3 Testing/Error Reporting ..... 22
**3.8 Suspend/resume Modem Services events from MCM for Tier 2 ..... 22**
3.8.1 Overview ..... 22
3.8.1.1 Platform Content Overview ..... 22
3.8.1.2 Product Expectations ..... 22
3.8.1.3 Extending Platform Functionality ..... 22
3.8.2 Setup ..... 22
3.8.2.1 Tools/Equipment Needed ..... 22
3.8.2.2 Setup Instructions ..... 22
3.8.2.3 Setup Troubleshooting ..... 22
3.8.3 Usage ..... 22
3.8.3.1 Basic Instructions ..... 22
3.8.3.2 Detailed Instructions ..... 22
3.8.3.3 Code Example ..... 23
3.8.3.4 Testing/Error Reporting ..... 25
**3.9 Get Modem Configuration File Count ..... 25**
3.9.1 Overview ..... 25
3.9.1.1 Platform Content Overview ..... 25
3.9.1.2 Product Expectations ..... 25
3.9.1.3 Extending Platform Functionality ..... 25
3.9.2 Setup ..... 25
3.9.2.1 Tools/Equipment Needed ..... 25
3.9.2.2 Setup Instructions ..... 25
3.9.2.3 Setup Troubleshooting ..... 25


© Continental Automotive GmbH 2019
Confidential and Proprietary

3.9.3 Usage ..... 25
3.9.3.1 Basic Instructions ..... 25
3.9.3.2 Detailed Instructions ..... 25
3.9.3.3 Testing/Error Reporting ..... 26
**4 Known Issues ..... 26**
**5 FAQs ..... 26**
**6 Troubleshooting/Tips and Tricks ..... 26**
**7 References ..... 26**
**7.1 Abbreviations/Acronyms/Definitions ..... 26**
**7.2 Platform Documentation ..... 27**
**7.3 Website links ..... 27**
**8 Revision History ..... 28**


© Continental Automotive GmbH 2019
Confidential and Proprietary

# 1 Introduction

This document provides usage information for Modem Configuration Manager, or simply MCM. The target audience for this document is developers and architects who will use the Telematics Platform MCM services.

MCM provides APIs to manage modem configuration files.

MCFG files contains instructions for modifying specific EFS items/files, there are two main types of MCFG files:

- Board Configuration Files: Commonly referred as Hardware mbn files, these files contain configurations that will affect feature (e.g. eCall, Audio etc).
- Carrier Configuration Files: Commonly referred as Software mbn files, these files contain carrier (AT&T, Vodafone etc) and region-specific configurations like APN settings, IMS and SRVCC capabilities etc.

MCM is a Linux Daemon that runs on the modem application processor and exposes public APIs that are invoked by user applications.

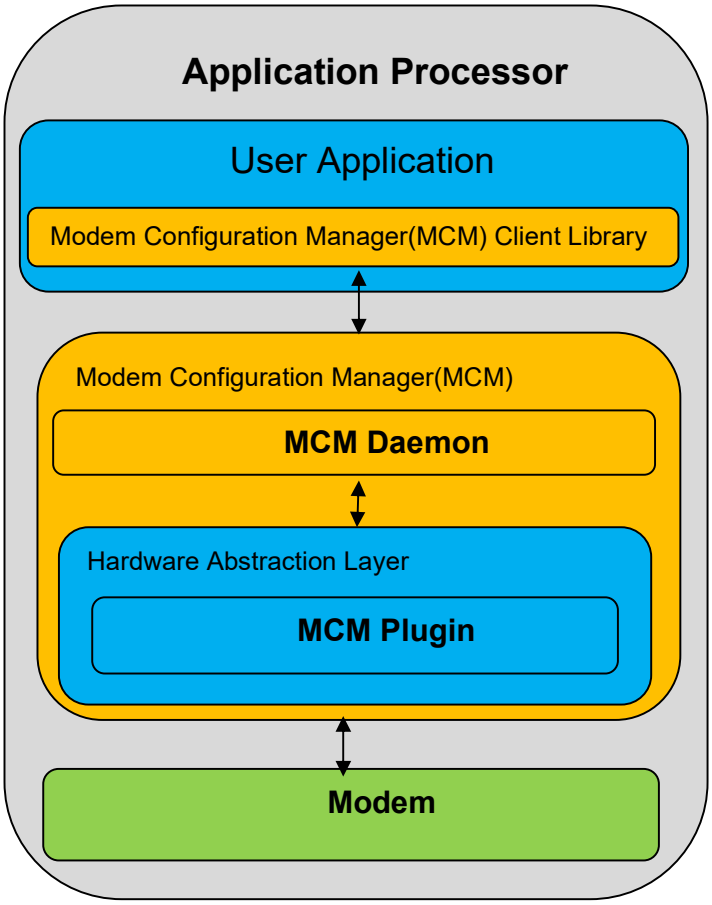


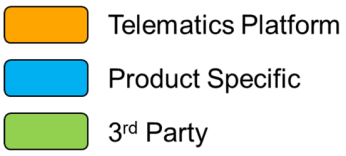

**Figure 1: MCM context diagram**


© Continental Automotive GmbH 2019
Confidential and Proprietary

## 1.1 Notational Conventions

| | |
|---|---|
| *Ctrl+x* | *A sequence with a '+' indicates that you must press and hold one key or pointing device button while pressing another key or pointing device button* |
| *[ ]* | *Optional elements in a command* |
| *…..* | *Indicates one of the following possibilities:*<br>• *Additional optional arguments in a statement have been omitted*<br>• *The preceding item or items can be repeated one or more times*<br>• *Additional parameters, values, or other information can be entered* |
| - | *Line continuation: indicates that a command line or code line continues on the following line when at the end of a line* |
| *Italic text* | *Indicates important information, complete titles of manuals or variables* |
| `Mono-spaced` | *Example code or interactive screen display/output* |
| *Numbers* | *All numbers are assumed to be decimal unless otherwise noted. Non decimal radixes – binary, octal or hexadecimal – are explicitly indicated* |
| Underline blue text | *A link to an external reference (e.g., website, document)* |
| `Modules`<br>`\|-> section`<br>`\|-> item` | *A link to a section or item in the Telematics Platform html interface specification (Doxygen output from platform code headers)* |

Diagram Legend




# 2 Setup

This section shall provide instructions for user of MCM also known as MCM client to set up a project capable of using services provided by MCM.

## 2.1 Tools/Equipment Needed

Not Applicable

## 2.2 Setup Instructions

1 The user of MCM must use the *MCM Client Library,* as this library is responsible for controlling communication between MCM Client application and MCM daemon, sending request and receiving synchronous responses.

1) Use pkg-config in the client application makefile to link MCM client, libraries.
`LDFLAGS += `pkg-config --libs mcm``

2) Include MCM Client Library header file in client application source code
`#include "mcm.h"`

3) MCM Client must call '`tpSYS_initIPC(E_SYS_IPC_DBUS, NULL, NULL)`' before registering with MCM.)

2 MCM Client application must be registered with MCM daemon to invoke any MCM APIs, see all details in Client registration


© Continental Automotive GmbH 2019
Confidential and Proprietary

### 2.2.1 Hardware Setup Instructions

Following are the setup instructions that MUST be followed for successful functioning of MCM's auto-selection functionality:

- Hardware MUST have proper RF calibration.
- Active SIM/eUICC properly inserted in SIM tray or soldered.
- Modem operating modem MUST NOT be in 'Factory Test Mode'.
- 'Persistency' MUST be installed (see TP_F101U-00026)

## 2.3 Setup Troubleshooting

Not Applicable

# 3 Features/Components

## 3.1 Client registration

### 3.1.1 Overview

#### 3.1.1.1 Platform Content Overview

This section describes how the application program can register and de-register a client with MCM.

After completing the Setup Instructions MCM client should call `'tpMCM_init()'` to start communication with MCM. MCM communicates with its clients and provides responses synchronously.

#### 3.1.1.2 Product Expectations

The following expectations must be met by application program to register or de-register as MCM client as designed:

1) To successfully register as MCM client application program must complete the Setup Instructions.
2) Application program should call `'tpMCM_init()'` API to register as MCM client ONLY after successful return from API call `'tpSYS_initIPC()'`.
3) ONLY upon successful return from `'tpMCM_init()'`, application program can call any other MCM APIs.

#### 3.1.1.3 Extending Platform Functionality

Not applicable

### 3.1.2 Setup

Not Applicable

#### 3.1.2.1 Tools/Equipment Needed

Not applicable

#### 3.1.2.2 Setup Instructions

Not applicable

#### 3.1.2.3 Setup Troubleshooting

Not applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

### 3.1.3 Usage

#### 3.1.3.1 Basic Instructions

Not applicable

#### 3.1.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to setup IPC | Modules<br>|-> Operating System (SYS)<br>|-> **tpSYS_initIPC()** |
| API to register an MCM client | Modules<br>|-> Modem<br>|-> Modem Configuration Manager(MCM)<br>|-> **tpMCM_init()** |
| Error types returned by MCM | Modules<br>|-> Modem<br>|-> Modem Configuration Manager(MCM)<br>|-> **tpMCM_Error_t** |

Before registering a new client, the application program must make sure that MCM is already registered before one of the following services in `tpsys.h` can be used (see TP_F101U-00023 Common Library user guide for more details).

To register as a client of MCM, the application program must call `tpMCM_init`().

Upon successful response from `tpMCM_init()`, application program can start using MCM APIs.

#### 3.1.3.3 Code Example

The following code example shows how the Application Program can use the APIs to register/de-register as MCM client:

**Code Example:** Register and De-Register a MCM Client

```
/* Includes */
#include "mcm.h"
#include "tpsys.h"
/*****************************************************************************
**          main
**
**          Performing:
**          - Register a client
**          - Create a thread to listen to the MCM
**          - De-register a client**
*****************************************************************************/
int main (int argc, char* const argv[])
{
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
        tpSYS_ReturnCode_t eSYSReturn = E_SYS_IPC_RET_FAILURE;
        tpMCM_Error_t err = E_MCM_ERROR_NONE;

        /* Setup IPC
        Note: It only needs to be called if it was not initialized in the NADIF
        client process yet */
        eSYSReturn = tpSYS_initIPC(E_SYS_IPC_DBUS, NULL,NULL);
        if(eSYSReturn == E_SYS_IPC_RET_FAILURE)
        {
              /* Unable to setup IPC */
              /* MCM client will print corresponding eSYSReturn error code */
              return 1;
        }
        else{
           /* Initialize MCM */
           err = tpMCM_init();
           if(err != E_MCM_ERROR_NONE)
          {
              /* Unable to initialize modem config manager (MCM)*/
              /* MCM client will print corresponding err code */
              return 1;
           }
        }
        {
         /*User application make calls to MCM*/
        }

     /* De-Init IPC setup */
     tpSYS_deinitIPC();
     return 0;
}
```

#### 3.1.3.4 Testing/Error Reporting

Not Applicable

## 3.2 Auto-selection of Modem Configurations

### 3.2.1 Overview

#### 3.2.1.1 Platform Content Overview

MCM and Modem performs automatic activation of appropriate Board(HW MBN) and Carrier (SW MBN) files respectively.. This activation procedure can be performed depending on region-specific carrier settings and this process is called 'auto-selection'. This will configure Modem Configuration files without intervention of User Application. However, if the User Application decides to use specific configuration file there are APIs provided (Refer Override Modem Configuration) to override auto-selection functionality and activate specific configuration as required.


© Continental Automotive GmbH 2019
Confidential and Proprietary

***Note: Once the auto-selection is disabled or enabled it is persistent across power cycles.***

Auto-selection is performed during initial flashing procedure when board is moved out of 'Factory Test Mode'. When board is in 'Factory Test Mode' none of Modem Configuration files will be activated. Automatic selection of 'Carrier configuration (SW MBN) is dependent on SIM/eUICC's ICCID (Region specific). For example, when AT&T SIM card is inserted into the Hardware, Modem shall activate AT&T specific 'carrier' configuration file subsequently appropriate 'carrier' configurations shall be activated depending on Roaming conditions. When none of the SIM/eUICC's ICCID matches then 'Rest of the World' (ROW) carrier configuration shall be activated.

***Note: 'Board configuration' (Hardware MBN) activation will cause SoC reset.***

#### 3.2.1.2 Product Expectations

For successful functioning of Auto-selection, Hardware should meet the pre-requisites as mentioned.

#### 3.2.1.3 Extending Platform Functionality

Not Applicable

### 3.2.2 Setup

#### 3.2.2.1 Tools/Equipment Needed

Not Applicable

#### 3.2.2.2 Setup Instructions

Not Applicable

#### 3.2.2.3 Setup Troubleshooting

Not Applicable

### 3.2.3 Usage

#### 3.2.3.1 Basic Instructions

Not applicable

#### 3.2.3.2 Detailed Instructions

Not applicable

#### 3.2.3.3 Testing/Error Reporting

Not applicable

## 3.3 Get Modem configuration List

### 3.3.1 Overview

#### 3.3.1.1 Platform Content Overview

MCM provides APIs to query the list of configuration files that are loaded on the Modem. This list will include details like 'name', 'version information', 'status of configuration file (active/inactive)'.

#### 3.3.1.2 Product Expectations

Not Applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

#### 3.3.1.3 Extending Platform Functionality

Not Applicable

### 3.3.2 Setup

Not Applicable

#### 3.3.2.1 Tools/Equipment Needed

Not Applicable

#### 3.3.2.2 Setup Instructions

Not Applicable

#### 3.3.2.3 Setup Troubleshooting

Not Applicable

### 3.3.3 Usage

#### 3.3.3.1 Basic Instructions

Not Applicable

#### 3.3.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to get list of Modem Configuration files loaded on Modem. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_getConfigList()** |

The API **tpMCM_getConfigList()** allows MCM clients to query the list of Modem configuration files which is loaded on the Modem.

This API returns tpMCM_Error_t - E_MCM_ERROR_NONE value if success.

and returns the available tpMCM_Config_t list of modem configuration files with "Name", "config Version" and "status active/inactive" and the config_list_size its size of the list returned.

#### 3.3.3.3 Code Example

The following code example shows how this API allows MCM client to get both Board configuration(HW MBN) and carrier (SW MBN) active and loaded configurations.

**Code Example**: GetConfigurationList

```
/* Includes */
#include "mcm.h"
#include "tpsys.h"
/*****************************************************************************
**          Get Modem Configuration Lists
**          Performing:
**          - Get active and loaded modem configuration lists
**
*****************************************************************************/
void GetConfigurationList()
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
{
   tpMCM_Error_t err = E_MCM_ERROR_NONE;
  tpMCM_Config_t *config_list[MCM_CONFIGS];
  guint8 idx = 0;
  guint8 config_list_size=0;

  for (idx = 0; idx < MCM_CONFIGS; idx++) {
    config_list[idx] = NULL;
  }
  err = tpMCM_getConfigList(config_list, &config_list_size);
  if (err != E_MCM_ERROR_NONE)
  {
   //Couldn't get config list for client
   /* MCM client will print corresponding err code*/
    return;
  } else {
     if (0 == config_list_size) {
        //No modem configurations loaded
        return;
     } else {
        for (idx = 0; idx < config_list_size; idx++) {
              /*Config Name:config_list[idx]->name*/
              /*Config version:config_list[idx]->config_version*/
              /*Config state:config_list[idx]->active */
        }
     }
  }
  return;
}
```

#### 3.3.3.4 Testing/Error Reporting

Not Applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

## 3.4 Delete Modem Configuration

### 3.4.1 Overview

#### 3.4.1.1 Platform Content Overview

MCM provides functionality to delete the Modem Configuration Files which are loaded on the Modem file system and Application processor (A7). When delete functionality is used MCM will delete configurations loaded on both Modem and Application processor.

**Note: Delete operation will delete activated configurations.**

#### 3.4.1.2 Product Expectations

**Application users should handle the delete operations carefully if configuration/s are deleted there is no way to restore back the original configuration data.**

**It is recommended by platform to make use of Software loading support to Update/Delete/Add the configurations.**

#### 3.4.1.3 Extending Platform Functionality

Not Applicable

### 3.4.2 Setup

Not Applicable

#### 3.4.2.1 Tools/Equipment Needed

Not Applicable

#### 3.4.2.2 Setup Instructions

Not Applicable

#### 3.4.2.3 Setup Troubleshooting

Not Applicable

### 3.4.3 Usage

#### 3.4.3.1 Basic Instructions

Not Applicable

#### 3.4.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to delete Modem Configuration files loaded on Modem. | `Modules`<br>`  \|-> Modem`<br>`  \|-> Modem Configuration Manager(MCM)`<br>`  \|-> tpMCM_deleteConfiguration()` |

The API **tpMCM_deleteConfiguration()** allows MCM clients to delete Modem configuration files which is loaded on the Modem.

This API returns tpMCM_Error_t - E_MCM_ERROR_NONE value if success.

#### 3.4.3.3 Code Example

The following code example shows how this API allows MCM client to delete the specific Modem configuration loaded files.


© Continental Automotive GmbH 2019
Confidential and Proprietary

**Code Example**: DeleteConfiguration

```
/* Includes */
#include "mcm.h"
#include "tpsys.h"
/*****************************************************************************
**          Delete Modem Configuration
**          Performing:
**          - Delete the modem configuration files which is already loaded.
**          Parameters:
**          @param[out] config - the config to be removed
*****************************************************************************/
void DeleteConfiguration(gchar *config){
     tpMCM_Error_t err = E_MCM_ERROR_NONE;
     //Delete the given modem configuration file
     err = tpMCM_deleteConfiguration(config);
     if( err != E_MCM_ERROR_NONE)
     {
          /*MCM is unable to delete the requested MCFG files*/
          /* MCM client will prints corresponding err code */
     }
     else {
         // removed from EFS and Persistency
     }
   return;
  }
```

#### 3.4.3.4 Testing/Error Reporting

Not Applicable

## 3.5 Manual Modem Configuration selection

### 3.5.1 Overview

**NOTE: Usage of Manual Modem Configuration is strictly for Continental internal certification purposes ONLY.**

#### 3.5.1.1 Platform Content Overview

MCM provides support for Manual Modem Configuration selection also known as override configuration., i.e. MCM clients can make use of MCMs APIs to manually configure required Hardware and Software MBNs (MCM allows activation of only single hardware and software MBN at any time).

When Manual Modem Configuration is active auto-selection will be disabled which is persistent across reboots until Manual Modem Configuration selection is disabled., as soon as override is disabled MCM's auto-selection logic shall activate corresponding Hardware and Software MBNs as per Hardware and SIM configuration.

MCM clients can query for current configuration selection state of MCM i.e. if auto-selection is active or not.


© Continental Automotive GmbH 2019
Confidential and Proprietary

#### 3.5.1.2 Product Expectations

**NOTE: Usage of Manual Modem Configuration is strictly for Continental internal certification purposes ONLY. For Product development teams, it is NOT recommended to use Manual Modem Configuration Selection. Using Manual Modem Configuration Selection shall have impact on Modem network connection causing issues related to permanent loss of network, IMS connection issues etc.**

#### 3.5.1.3 Extending Platform Functionality

Not Applicable

### 3.5.2 Setup

#### 3.5.2.1 Tools/Equipment Needed

Not Applicable

#### 3.5.2.2 Setup Instructions

Not Applicable

#### 3.5.2.3 Setup Troubleshooting

Not Applicable

### 3.5.3 Usage

#### 3.5.3.1 Basic Instructions

The following are the fundamental aspects of this API:

1) MCM Clients can use `tpMCM_setOverride()` API to request the MCM to manual override the state of modem configuration override over existing one.
2) MCM Clients can use `tpMCM_getOverrideState()` API to get the state of the modem configuration override.
3) MCM Clients can use `tpMCM_OverrideDisable()` API to request the MCM to disable the modem configuration override.

#### 3.5.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to request to override the Modem Configuration files loaded on Modem. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_setOverride()** |
| API to request to get the state of the Modem Configuration override. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_getOverrideState()** |
| API to request to disable modem configuration override | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_OverrideDisable()** |
| Enumeration of error response codes returned by MCM | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager (MCM)<br>\|-> **tpMCM_Error_t** |


© Continental Automotive GmbH 2019
Confidential and Proprietary

When using **tpMCM_setOverride()**, MCM clients MUST send the complete list of configurations by marking the each configuration is with set for activation flag as TRUE or FALSE. Please refer following coding sample for better understanding.

Upon successful execution of override, auto-selection functionality will be DISABLED and is persistent across reboots until override functionality is disabled by using **tpMCM_OverrideDisable()**.

**Code Example**: setOverride

```
/* Includes */
#include "mcm.h"
#include "tpsys.h"
/*****************************************************************************
**          Manual Modem Configuration Selection (Override configuration)
**          Performing:
**          - Manual Override of active Hardware and Software to different
Configuration.
**          Parameters:
**          @param[IN] tpMCM_Config_t** config - config list
**          @param[OUT] tpMCM_Error_t - MCM error return.
*****************************************************************************/
void setOverride(tpMCM_Config_t** config)
{
     tpMCM_Error_t error = E_MCM_ERROR_NONE;
     tpMCM_Config_t *new_config_list_request[MCM_CONFIGS];
     tpMCM_Config_t *current_config_list[MCM_CONFIGS];
     guint8 config_list_size=0;
     guint8 idx = 0;

      /*MCM clients must first query all the available Modem configurations that
      are loaded on the Modem., this shall let MCM client know number of
      available configurations loaded on Modem along with configuration names*/

      error= tpMCM_getConfigList(current_config_list, &current_config_list_size);
      if(E_MCM_ERROR_NONE != err)
      {
          printf("Couldn't get config list for client, error %d\n", err);
      }
      else
      {
         if(config_list_size == 0)
         {
           printf ("No modem configurations loaded\n");
         }
      }
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
      /*MCM clients must manage memory of configuration list., MCM shall reject
      the request with 'E_MCM_ERROR_INVALID_ARGUMENT' if empty list is received
      from MCM clients., Memory allocation for list must be with size of
      configurations received when config list is queried*/

      for (idx = 0; idx < config_list_size; idx++)
      {
        /*Allocate memory for 'list_request' and init the new config list
      'tpMCM_Config_t' with default values and configuration names fetched from
      above request*/

          list_request[idx] = (tpMCM_Config_t * ) malloc(sizeof(tpMCM_Config_t));
          if (NULL == new_config_list_request[idx]) {
             printf("No memory");
             return;
          }
            memset(new_config_list_request [idx], 0, sizeof(tpMCM_Config_t));
            /*Initialize the request list with default values*/
            g_strlcpy(new_config_list_request[idx]->name,
                      current_config_list[idx]->name,
                      sizeof(list_request[idx]->name));
                      list_request[idx]->active = FALSE;
           }
      }

      /*MCM clients based on upon their criteria be it user selection has to
      update the new activation list with configuration that must are required
      for new modem configuration selection.
      Ex: if modem has HW_CONFIG_MBN and ATT_SW_MBN, CMCC_SW_MBN loaded and
      active ones are HW_CONFIG_MBN and ATT_SW_MBN while CMCC_SW_MBN is
      inactive., If MCM clients has to update the configuration for CMCC_SW_MBN
      has to mark CMCC_SW_MBN as TRUE and ATT_SW_MBN as FALSE before calling
      setOverride API. Note that HW_CONFIG_MBN activation flag MUST be TRUE*/

       error = tpMCM_setOverride(new_config_list_request);
       if (E_MCM_ERROR_NONE != error)
       {
          printf("Couldn't set override for client, error %d\n", error);
       }
    return;
  }
```

A value of E_MCM_ERROR_NONE will be returned in case of success. In case of failure a value indicative of the failure type tpMCM_Error_t will be returned.


© Continental Automotive GmbH 2019
Confidential and Proprietary

For an extensive list of error codes, list of error codes, see `tpMCM_Error_t` type definition. The following table explains some of these error codes in the context of MCM APIs error types returned

| Error Code | Scenario |
|---|---|
| `E_MCM_ERROR_NONE` | MCM API returns with success |
| `E_MCM_ERROR_INVALID_ARGUMENT` | MCM API received a bad parameter from caller |
| `E_MCM_ERROR_INTERNAL` | MCM is in incorrect state |
| `E_MCM_ERROR_NO_MEMORY` | MCM fails to allocate memory dynamically. |

#### 3.5.3.3 Testing/Error Reporting

Not Applicable

## 3.6 Software Loading Support

### 3.6.1 Overview

#### 3.6.1.1 Platform Content Overview

MCM provides support for updating Modem Configuration Files (Both hardware and software MBN files) during software update campaigns. For example, Modem configuration that controls IMS behavior is updated by Platform it is required to roll out the new configuration to already configured hardware. MCM as an update agent supports Adding/Updating/Deleting existing Modem Configurations via Software loading. During the software activation verification phase, the name and version number of each Modem Configuration File are verified, including inactive files.

Refer TP_F101U-00028

#### 3.6.1.2 Product Expectations

Product teams are expected to create MCFG package with tar.gz extension, which holds updated MCFG files and MCFG file name and version information text file. This package needs to be part of Software loading package.

Refer TP_F101U-00028

#### 3.6.1.3 Extending Platform Functionality

Not Applicable

### 3.6.2 Setup

#### 3.6.2.1 Tools/Equipment Needed

Not Applicable

#### 3.6.2.2 Setup Instructions

Not Applicable

#### 3.6.2.3 Setup Troubleshooting

Not Applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

### 3.6.3 Usage

#### 3.6.3.1 Basic Instructions

Not Applicable

#### 3.6.3.2 Detailed Instructions

The modem configuration package is generated as part of the build:

```
./release/images/devel/4K/tp_modem_config_list.tar.gz
```

#### 3.6.3.3 Testing/Error Reporting

Not Applicable

## 3.7 Get MCFG state change information

### 3.7.1 Overview

#### 3.7.1.1 Platform Content Overview

MCM provides functionality to request current MCFG state and obtain unsolicited MCFG state change notifications via a registered callback. Supported MCFG states:

- MCFG activation happened (HW or SW mbn)
- MCFG installation in progress
- MCFG installation completed
- MCFG installation failed (0KB size files found, authentication failure, corrupted tarball detected, file extension: not tar.gz, no.mbn)
- MCFG activation for SW only MBN (Modem Reset is required to activate SW MBN)

#### 3.7.1.2 Product Expectations

- When the MCFG state changes to `E_MCM_MCFG_SW_ONLY_CONFIG_REBOOT_REQUIRED`, application needs to reset the modem to activate SW MBN.
- When a MCFG update is performed, applications that include custom API changes should wait for MCFG state change to `E_MCM_MCFG_SW_ACTIVATED` before updating custom APN settings. `E_MCM_MCFG_SW_ACTIVATED` state indicates that no further automatic APN changes will be made as a result of the MCM update.

#### 3.7.1.3 Extending Platform Functionality

Not Applicable

### 3.7.2 Setup

#### 3.7.2.1 Tools/Equipment Needed

Not Applicable

#### 3.7.2.2 Setup Instructions

Not Applicable

#### 3.7.2.3 Setup Troubleshooting

Not Applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

### 3.7.3 Usage

#### 3.7.3.1 Basic Instructions

The following expectations must be met by application program to receive unsolicited MCFG state change notifications:

i. Application program should invoke `tpMCM_RegisterConfigNotify()` API to register a callback function for receiving MCFG state change notifications.
ii. Application program should invoke `tpMCM_DeregisterConfigNotify()` to de-register the callback function and stop receiving unsolicited MCFG state change notifications.

The following expectations must be met by application program to obtain current MCFG state synchronously:

i. Application program should invoke `tpMCM_getConfigurationUpdateStatus()` API to get the current MCFG state.

#### 3.7.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to register callback function to receive unsolicited MCFG state information. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_RegisterConfigNotify** |
| API to deregister from receiving unsolicited MCFG state information. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_DeregisterConfigNotify** |
| Typedef of callback function for receiving unsolicited MCFG state information | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_mcfg_resp_notify_t** |
| API to get the current MCFG state information. | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_getConfigurationUpdateStatus** |
| MCFG state enumeration | Modules<br>\|-> Modem<br>\|-> Modem Configuration Manager(MCM)<br>\|-> **tpMCM_mcfg_status_t** |

These API returns tpMCM_Error_t - E_MCM_ERROR_NONE value if success.


© Continental Automotive GmbH 2019
Confidential and Proprietary

#### 3.7.3.3 Testing/Error Reporting

Not Applicable

## 3.8 Suspend/resume Modem Services events from MCM for Tier 2

### 3.8.1 Overview

#### 3.8.1.1 Platform Content Overview

The MCM manager will enable client to suspend/resume modem services through the suspend/resume API call.

#### 3.8.1.2 Product Expectations

The following expectations must be met by application program to suspend or resume the modem services:

1) To successfully suspending/resuming the Modem services application program must complete the Setup Instructions.
2) Application program should call '`tpMCM_suspendResumeModemServices()`' API to suspend/resume the Modem services in MCM ONLY after successful return from API call '`tpSYS_initIPC()`'.

#### 3.8.1.3 Extending Platform Functionality

Not applicable

### 3.8.2 Setup

Not Applicable

#### 3.8.2.1 Tools/Equipment Needed

Not applicable

#### 3.8.2.2 Setup Instructions

Not applicable

#### 3.8.2.3 Setup Troubleshooting

Not applicable

### 3.8.3 Usage

#### 3.8.3.1 Basic Instructions

Not applicable

#### 3.8.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to get access to suspend/resume feature | `Modules`<br>`  \|-> Modem`<br>`  \|-> Modem Configuration Manager(MCM)`<br>`  \|-> tpMCM_suspendResumeModemServices` |
| Modem events state types returned by MCM | `Modules`<br>`  \|-> Modem`<br>`  \|-> Modem Configuration Manager(MCM)`<br>`  \|-> tpMCM_Modem_Service_State_t` |


© Continental Automotive GmbH 2019
Confidential and Proprietary

The API 'tpMCM_suspendResumeModemServices' has a parameter whose input will be provided by the client.There are two available states:

- E_MCM_SUSPEND_MODEM_EVENTS – invoke suspend MCM services (UIM, DMS, PDC, CUST QMI)
- E_MCM_RESUME_MODEM_EVENTS – resumes services from the suspended states

To suspend/resume the Modem services , the application program must call the API function `tpMCM_suspendResumeModemServices`().

Upon successful response from `tpMCM_suspendResumeModemServices()`, application program has been disable the Modem services in suspension case or run it in resuming case.
A value of E_MCM_ERROR_NONE will be returned in case of success. In case of failure a value indicative of the failure type tpMCM_Error_t will be returned.

For an extensive list of error codes, list of error codes, see `tpMCM_Error_t` type definition. The following table explains some of these error codes in the context of MCM APIs error types returned

| Error Code | Scenario |
|---|---|
| `E_MCM_ERROR_NONE` | MCM API returns with success |
| `E_MCM_ERROR_INVALID_ARGUMENT` | MCM API received a bad parameter from caller |
| `E_MCM_ERROR_INTERNAL` | MCM is in incorrect state |
| `E_MCM_ERROR_NO_MEMORY` | MCM fails to allocate memory dynamically. |

#### 3.8.3.3 Code Example

The following code example shows how the Application Program can use the APIs to suspend/resume MCM services:

**Code Example:** Register and De-Register a MCM Client

```
/* Includes */
#include "mcm.h"
#include "tpsys.h"
/*****************************************************************************
**
**          Performing:
**          - invoke the function to change the QMI modem status Parameters:
**         @param[IN] tpMCM_Modem_Service_State_t state - state of action - suspend
**         or resume Modem Services
**         @param[OUT] tpMCM_Error_t - MCM error return.
*****************************************************************************/
static void tpMCM_TEST_suspendResumeModemServices(gboolean is_command_line, gchar*
value)
{
    tpMCM_Error_t err = E_MCM_ERROR_NONE;
    gchar params[PARAMS_SIZE][BUFFER_SIZE] = {'\0'};
    gchar * input_data = NULL;
    gboolean choice = TRUE, input_valid = FALSE;
    tpMCM_Modem_Service_State_t mcm_mdm_srv_state = E_MCM_SUSPEND_MODEM_EVENTS;

    if(TRUE == is_command_line){
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
        input_data = strtok(value,",");
        if (NULL == input_data) {
            MCM_TEST_LOG_ERR("Invalid option in the command line\n");
        } else {
          choice = atoi(input_data);
        }

      } else {
        fprintf(stdout, "Enter the option 0 to suspend or 1 to resume all modem
services");
        tpMCM_TEST_get_input_data(params[0], 100);
        input_data = strtok(params[0], ",");
        if (NULL == input_data) {
            MCM_TEST_LOG_ERR("Invalid option chosen\n");
        } else {
          choice = atoi(input_data);
        }
      }

    if(E_MCM_SUSPEND_MODEM_EVENTS == choice){
        mcm_mdm_srv_state = E_MCM_SUSPEND_MODEM_EVENTS;
        input_valid = TRUE;
        MCM_TEST_LOG_INFO("User intended to stop all modem services");
    }
    else if (E_MCM_RESUME_MODEM_EVENTS == choice){
        mcm_mdm_srv_state = E_MCM_RESUME_MODEM_EVENTS;
        input_valid = TRUE;
        MCM_TEST_LOG_INFO("User intended to resume all modem services");
    }
    else{
        MCM_TEST_LOG_ERR("User entered a wrong value other than 0 or 1, please try
again");
    }

    if(TRUE == input_valid){
        err = tpMCM_suspendResumeModemServices(mcm_mdm_srv_state);
        if(E_MCM_ERROR_NONE != err){
           MCM_TEST_LOG_ERR("MCM Test : Couldn't suspend resume modem services, err
%u\n", err);
        } else {
        MCM_TEST_LOG_INFO("Your request has been taken up");
        }
    }
```


© Continental Automotive GmbH 2019
Confidential and Proprietary

```
}
```

#### 3.8.3.4 Testing/Error Reporting

Not Applicable

## 3.9 Get Modem Configuration File Count

### 3.9.1 Overview

#### 3.9.1.1 Platform Content Overview

MCM provides API to query the number of configuration files that are loaded on the Modem. The count value includes total number of SW MBN and HW MBN together irrespective whether the configuration is active or inactive.

#### 3.9.1.2 Product Expectations

Not Applicable

#### 3.9.1.3 Extending Platform Functionality

Not Applicable

### 3.9.2 Setup

Not Applicable

#### 3.9.2.1 Tools/Equipment Needed

Not Applicable

#### 3.9.2.2 Setup Instructions

Not Applicable

#### 3.9.2.3 Setup Troubleshooting

Not Applicable

### 3.9.3 Usage

#### 3.9.3.1 Basic Instructions

Not Applicable

#### 3.9.3.2 Detailed Instructions

| API/Type Description | Doxygen Path |
|---|---|
| API to get count of Modem Configuration files loaded on Modem. | `Modules`<br>`  \|-> Modem`<br>`  \|-> Modem Configuration Manager(MCM)`<br>`  \|-> tpMCM_getConfigCount()` |

The tpMCM_getConfigCount**()** allows MCM clients to query the count of Modem configuration files which is loaded on the Modem.


© Continental Automotive GmbH 2019
Confidential and Proprietary

This API returns tpMCM_Error_t - E_MCM_ERROR_NONE value if success, and returns config_count, the total number of modem configurations.

#### 3.9.3.3 Testing/Error Reporting

Not Applicable

# 4 Known Issues

Not Applicable

# 5 FAQs

Not Applicable

# 6 Troubleshooting/Tips and Tricks

Not Applicable

# 7 References

## 7.1 Abbreviations/Acronyms/Definitions

| | |
|---|---|
| API | Application Programming Interface |
| APN | Access Point Name |
| EFS | Modem Embedded file system |
| FTM | Modem Factory Test Mode |
| MCFG | Modem Configuration File |
| MCM | Modem Configuration Manager |
| SW MBN | Software Modem Binary |
| HW MBN | Hardware Modem Binary |
| QMI | Qualcomm Messaging Interface |


© Continental Automotive GmbH 2019
Confidential and Proprietary

## 7.2 Platform Documentation

TP_102G-00001: Telematics Platform Glossary
TP_F101U-00023: Common Library User Guide
TP_F101U-00026: Persistence User Guide
TP_F101U-00028: Software Loading Manager User Guider

## 7.3 Website links

Not Applicable


© Continental Automotive GmbH 2019
Confidential and Proprietary

# 8 Revision History

| Version | Date | Description |
|---|---|---|
| 01.00 | Feb 18, 2020 | Initial ConMod release |
| 02.00 | Apr 08, 2022 | Update to suspend/resume modem services |
| 03.00 | Feb 16, 2023 | Get Modem configuration List Count |


© Continental Automotive GmbH 2019
Confidential and Proprietary